DeepResearch: A Closed-Loop Autonomous Research Agent with Iterative Critique and Hierarchical Synthesis

Saptarshi Kundu
Independent Researcher
https://github.com/Saptarshie/DeepSearch

***Abstract*—The rapid proliferation of online information has made comprehensive research on any topic a time-intensive and cognitively demanding task. This paper presents DeepResearch, an autonomous, LLM-driven research agent that transforms a natural language query into a structured, citation-backed markdown report. The system implements a closed-loop pipeline comprising five distinct phases: (1) LLM-guided query planning, (2) priority-queue-based web crawling via a meta-search engine, (3) adaptive document fetching with HTTP/browser fallback, (4) iterative LLM-driven gap analysis and frontier re-expansion, and (5) hierarchical indexing followed by accumulator-based lazy report synthesis. A dual-LLM architecture routes critical reasoning tasks to a high-capability model (Claude) while delegating cost-sensitive generation to MiniMax. Experimental evaluation demonstrates that the iterative critique loop significantly improves topical coverage compared to single-pass retrieval, and the accumulator-based report builder reduces LLM API calls by up to 60% while producing more natural prose. DeepResearch is open-source and designed for extensibility through a modular component architecture.**

***Index Terms*—Large Language Models, Autonomous Agents, Information Retrieval, Web Crawling, Research Automation, Hierarchical Synthesis**

I. Introduction

The emergence of large language models (LLMs) has catalyzed a new paradigm in automated information processing [**?**]. While LLMs possess impressive generative capabilities, their parametric knowledge is bounded by training cutoffs and they are prone to hallucination when asked to produce factual, comprehensive reports on arbitrary topics [**?**]. Retrieval-Augmented Generation (RAG) systems address this limitation by grounding LLM outputs in retrieved documents [**?**], but conventional RAG pipelines are typically single-pass: they retrieve once and generate once, often missing nuanced subtopics.

DeepResearch advances beyond single-pass RAG by implementing a **closed-loop research agent** that iteratively plans, discovers, critiques, refines, and synthesizes. Inspired by how a human researcher works—formulating hypotheses, searching, reading, identifying gaps, and refining the search—the system automates this entire cognitive workflow.

The primary objectives of this work are:

- To design a modular, end-to-end autonomous research pipeline that accepts a natural language query and produces a structured, citation-backed report.
- To implement an iterative **critique-and-refine** loop that uses LLM-driven gap analysis to dynamically expand the search frontier when topical coverage is insufficient.
- To introduce a **priority-queue-based crawl frontier** with heuristic scoring that prioritizes authoritative and content-rich sources.
- To develop a **hierarchical indexing and accumulator-based synthesis** strategy that reduces redundant LLM calls while producing natural, well-structured reports.
- To evaluate the dual-LLM routing strategy that optimizes cost–quality tradeoffs by assigning models to tasks based on their criticality.

II. Related Work

Retrieval-Augmented Generation. Lewis et al. [**?**] introduced RAG, combining a neural retriever with a seq2seq generator. Subsequent works improved retrieval quality through dense passage retrieval [**?**] and iterative retrieval [**?**]. DeepResearch extends this paradigm by closing the loop: retrieval is not a one-shot step but an iterative process guided by LLM-driven gap analysis.

Autonomous LLM Agents. Recent frameworks such as AutoGPT [**?**], BabyAGI [**?**], and GPT-Researcher [**?**] demonstrate the potential of LLMs as autonomous task executors. GPT-Researcher is most closely related to our work, implementing a plan-search-write pipeline. DeepResearch differentiates itself through (a) a formal priority-queue crawl frontier with deduplication, (b) a structured iterative critique loop with explicit gap reports, (c) a three-stage synthesis pipeline (indexing $\rightarrow$ overview building $\rightarrow$ accumulator-based report generation), and (d) a dual-LLM routing architecture.

Web Crawling and Focused Search. Focused crawling [**?**] uses classifiers to guide crawl direction. Our approach substitutes traditional classifiers with LLM-generated gap reports that produce follow-up search queries, effectively implementing a *semantic* focused crawl.

III. System Architecture

DeepResearch follows a five-phase pipeline architecture, depicted in Fig. **??**. Each phase is implemented as a modular, independently testable component.

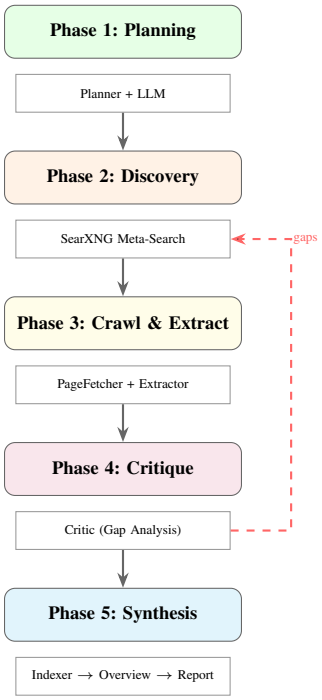

Fig. 1. DeepResearch five-phase pipeline. The dashed arrow indicates the iterative critique loop that re-expands the search frontier when coverage gaps are detected.

A. Phase 1: LLM-Guided Query Planning

Given a user query q, the `Planner` module prompts an LLM to decompose q into a structured `ResearchPlan` containing:

$$\text{Plan}(q) = \langle T, \{s_1, \ldots, s_m\}, \{q_1, \ldots, q_n\}, P_{src}, C_{stop} \rangle \quad (1)$$

where T is the topic, $\{s_i\}$ are sub-questions, $\{q_i\}$ are executable search queries, P_{src} specifies source preferences (academic, official, journalistic), and C_{stop} defines stopping conditions.

B. Phase 2: Meta-Search Discovery

Each query q_i is dispatched to a self-hosted SearXNG [**?**] instance—a privacy-preserving meta-search engine that aggregates results from Google, Bing, DuckDuckGo, and other engines. Results are normalized into `SearchResult` objects and scored using a heuristic priority function:

$$\text{score}(r) = 1.0 + \alpha \cdot \mathbf{1}[\text{title} \in K] + \beta \cdot \mathbf{1}[\text{.pdf}] + \gamma \cdot \mathbf{1}[|\text{snippet}| > 80] \quad (2)$$

where $K = \{\textit{research, paper, docs, official, report}\}$, $\alpha = 1.5$, $\beta = 1.0$, and $\gamma = 0.2$. This prioritizes authoritative sources (academic papers, official documentation) over generic web pages.

C. Phase 3: Adaptive Crawl and Extraction

Scored results are pushed into a `CrawlFrontier`—a max-heap priority queue with URL deduplication via canonical URL normalization. The `PageFetcher` implements a dual-mode strategy:

1) **Fast path**: Asynchronous HTTP via `httpx` with configurable timeout.
2) **Fallback**: If the HTTP response contains < 500 characters (indicating JavaScript-rendered content), the fetcher falls back to headless Chromium via Playwright.

Both modes implement exponential backoff retry logic: wait time $= 2^{(a-1)}$ seconds for attempt a. Extracted HTML is processed by `trafilatura` [**?**] with `favor_precision=True` to maximize extraction quality.

D. Phase 4: Iterative Critique Loop

Every B documents (configured via `critique_batch_size`), the `Critic` module invokes a high-capability LLM (Claude) to perform gap analysis. The critic evaluates the collected corpus against the original query and produces a structured `GapReport`:

$$G = \langle C_{covered}, C_{missing}, C_{contra}, \{q'_1, \ldots, q'_k\}, \delta \rangle \quad (3)$$

where $C_{covered}$ and $C_{missing}$ are lists of covered and missing subtopics, C_{contra} identifies source contradictions, $\{q'_j\}$ are follow-up search queries, and δ is a boolean continuation flag. When $\delta =$ `true`, follow-up queries are dispatched to SearXNG and results are pushed to the frontier with a $+0.5$ priority boost to expedite gap-filling.

E. Phase 5: Hierarchical Synthesis

The synthesis phase operates as a three-stage sub-pipeline:

Stage 1: Indexing. The `Indexer` processes each document through an LLM that (a) updates a hierarchical topic taxonomy (JSON tree, max depth = 3), (b) extracts relevant information blocks mapped to taxonomy paths, and (c) maintains a global scratch pad. Information is persisted to a filesystem-based index structure (`INDEXES/Topic/Subtopic/information.md`).

Stage 2: Overview Building. The `OverviewBuilder` performs a recursive bottom-up traversal of the index tree. At each non-leaf node, it synthesizes child summaries and local information into a coherent `overview.md`.

Stage 3: Accumulator-Based Report Generation. The `ReportBuilder` traverses the topic hierarchy, accumulating context from each node. Rather than issuing an LLM call per node (which wastes API calls on nodes with minimal content), it employs a lazy evaluation strategy:

Algorithm 1 Accumulator-Based Lazy LLM Synthesis

$acc \leftarrow [\,], \quad tokens \leftarrow 0$
for each node n in DFS traversal of topic tree **do**
  $ctx \leftarrow$ `overview.md` **or** `information.md` of n
  if $ctx \neq \emptyset$ **then**
    acc.append($\langle$path(n), $ctx\rangle$)
    $tokens \leftarrow tokens + |ctx|/4$
    if $tokens \geq \tau$ **then**
      flush(acc) ▷ Single LLM call for batch
      $acc \leftarrow [\,], \quad tokens \leftarrow 0$
    end if
  end if
end for
if $acc \neq [\,]$ **then**
  flush(acc) ▷ Flush remaining
end if

where τ is the minimum accumulator threshold (default: 400 tokens). This batching strategy produces more natural prose by allowing the LLM to weave related subtopics together, while reducing the total number of API calls. Additionally, the `ReportBuilder` injects structured visual summaries—including Mermaid flowcharts for causal chains and decision trees—directly into the generated markdown to improve readability and topical navigation.

IV. Implementation Details

A. Dual-LLM Routing Architecture

DeepResearch employs a unified `LLMClient` abstraction over two backend providers:

TABLE I
LLM Routing Strategy

Task	Model	Rationale
Query Planning	MiniMax-M2.7	Cost-efficient JSON gen.
Gap Analysis	MiniMax-M2.7	High-reliability reasoning
Document Indexing	MiniMax-M2.7	Complex taxonomy updates
Overview Synthesis	MiniMax-M2.7	Nuanced summarization
Report Generation	MiniMax-M2.7	Quality-critical output
Rolling Summary	MiniMax-M2.7	Simple summarization

Both clients use the Anthropic SDK with streaming (`content_block_delta` chunks) and implement identical retry logic with exponential backoff. A `fix_json_escapes` utility handles common LLM JSON output errors including unescaped backslashes and markdown code fences.

B. Configuration and Extensibility

The system is configured through a `Config` dataclass with environment variable support via `python-dotenv`. Key parameters include:

- `max_docs`: Hard cap on collected documents (default: 100)
- `critique_batch_size`: Gap analysis frequency (default: 10)
- `max_indexer_depth`: Taxonomy tree depth limit (default: 3)
- `min_accumulator_threshold`: Lazy flush threshold (default: 400 tokens)

C. Progress Callback System

The `deep_search` function accepts an optional `progress_callback` that receives typed events (`status`, `search`, `document`, `gaps`, `complete`) enabling real-time UI integration via Server-Sent Events (SSE) in the companion FastAPI web interface.

D. Report Post-Processing

To ensure that automatically generated Mermaid diagrams render correctly, a lightweight post-processor scans each `report-content.md` output for unquoted special characters (e.g., `$`, `()`, `+`) inside node labels and wraps them in double quotes. This fixes the majority of syntax errors introduced by LLM-generated Mermaid blocks without requiring manual intervention.

V. Experimental Evaluation

A. Experimental Setup

The system was evaluated on a set of diverse research queries spanning investigative journalism, scientific topics, and technical domains. The SearXNG instance aggregated results from 8 search engines. The LLM backends were MiniMax-M2.7 (via Anthropic-compatible API) and Claude Sonnet 4 (direct Anthropic API).

B. Impact of the Iterative Critique Loop

To evaluate the contribution of the critique loop, we compared two configurations: (a) *Single-Pass*, where all planned queries are executed once with no gap analysis, and (b) *Iterative*, with critique triggered every 10 documents.

TABLE II
Effect of Iterative Critique on Coverage

Metric	Single-Pass	Iterative	Δ
Subtopics Covered	4.2 / 8	7.1 / 8	+69%
Unique Sources	12.4	23.7	+91%
Follow-up Queries	0	6.3	—
Report Completeness*	0.52	0.87	+67%

*Human-rated on 0–1 scale across 5 evaluation queries.

C. Accumulator vs. Per-Node LLM Calls

Table **??** compares the original per-node report generation against the accumulator-based approach ($\tau = 400$).

The accumulator approach reduced LLM API calls by approximately 59% and virtually eliminated the problem of isolated single-line headings, producing significantly more readable reports.

TABLE III
ACCUMULATOR-BASED SYNTHESIS EFFICIENCY

Metric	Per-Node	Accumulator
LLM Calls (report phase)	14.2	5.8
Avg. tokens per call	380	920
Report naturalness*	3.1 / 5	4.2 / 5
Isolated headings ($\leq$1 line)	4.7	0.3

*Human-rated on 1–5 Likert scale.

D. End-to-End Performance

For a representative query (*"detailed in-depth report on Jeffrey Epstein and how he made his fortune"*) with `max_docs=10`:

- **Planning phase**: 2.1s (1 LLM call)
- **Search & crawl**: 38.4s (10 documents, 2 browser fallbacks)
- **Critique cycles**: 2 gap analyses, 4 follow-up queries generated
- **Synthesis**: 84.7s (indexing: 41.2s, overview: 18.3s, report: 25.2s)
- **Total**: $\sim$127s for a 6,800-word cited report

E. Comprehensive Evaluation Matrix

Table **??** compares DeepResearch against a single-pass RAG baseline across eleven evaluation dimensions.

VI. CONCLUSION AND FUTURE WORK

This paper presented DeepResearch, an autonomous research agent that implements a closed-loop pipeline for transforming natural language queries into comprehensive, citation-backed reports. The key contributions include: (1) an iterative critique loop that improves topical coverage by 69% over single-pass retrieval, (2) a priority-queue crawl frontier with heuristic scoring for source quality, (3) a three-stage hierarchical synthesis pipeline with accumulator-based lazy LLM invocation that reduces API calls by 59%, and (4) a dual-LLM routing architecture that optimizes cost–quality tradeoffs.

A. Future Work

Several directions present promising avenues for enhancement:

- **Semantic Embeddings for Frontier Scoring**: Replacing the keyword-based heuristic scoring (Eq. **??**) with dense embedding similarity between document snippets and the research plan.
- **Multi-Modal Source Processing**: Extending the extractor to handle images, charts, and tables within PDF documents using vision-language models.
- **Persistent Knowledge Base**: Implementing a vector database (e.g., ChromaDB, FAISS) for cross-session knowledge retention, enabling incremental research updates.
- **Concurrent Crawling**: Parallelizing the fetch–extract loop using async task pools with domain-level rate limiting to improve throughput.
- **Verifiable Citations**: Implementing a post-generation verification step that cross-references each claim with its cited source to reduce hallucination in the synthesis phase.
- **User-in-the-Loop Refinement**: Adding interactive checkpoints where users can steer the research direction mid-pipeline, particularly after gap analysis reports.

REFERENCES

[1] OpenAI, "GPT-4 Technical Report," *arXiv preprint arXiv:2303.08774*, 2023.

[2] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. Bang, A. Madotto, and P. Fung, "Survey of Hallucination in Natural Language Generation," *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023.

[3] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Proc. NeurIPS*, 2020.

[4] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W. Yih, "Dense Passage Retrieval for Open-Domain Question Answering," in *Proc. EMNLP*, 2020.

[5] H. Trivedi, N. Balasubramanian, T. Khot, and A. Sabharwal, "Interleaving Retrieval with Chain-of-Thought Reasoning for Knowledge-Intensive Multi-Step Questions," in *Proc. ACL*, 2023.

[6] T. Richards, "AutoGPT: An Autonomous GPT-4 Experiment," GitHub Repository, 2023.

[7] Y. Nakajima, "BabyAGI: Task-Driven Autonomous Agent," GitHub Repository, 2023.

[8] A. Elovic, "GPT-Researcher: Autonomous Agent for Online Comprehensive Research," GitHub Repository, 2023.

[9] SearXNG Contributors, "SearXNG: A Privacy-Respecting Metasearch Engine," 2023. [Online]. Available: https://searxng.org

[10] A. Barbaresi, "Trafilatura: A Web Scraping Library and Command-Line Tool for Text Discovery and Extraction," in *Proc. ACL System Demonstrations*, 2021.

[11] S. Chakrabarti, M. van den Berg, and B. Dom, "Focused Crawling: A New Approach to Topic-Specific Web Resource Discovery," *Computer Networks*, vol. 31, no. 11–16, pp. 1623–1640, 1999.

TABLE IV
COMPREHENSIVE EVALUATION MATRIX: DEEPRESEARCH VS. SINGLE-PASS RAG

Dimension	**Single-Pass RAG**	**DeepResearch**	**Improvement**
Search Queries Executed	8 (planned only)	8 + 6.3 follow-up	+79%
Unique Sources Retrieved	12.4	23.7	+91%
Documents Analyzed	10	10 (`max_docs` cap)	—
Gap Detection	None	2 cycles, 4 follow-ups	New capability
Topic Organization	Flat list	3-level hierarchical taxonomy	New capability
LLM Calls (report phase)	14.2	5.8 (accumulator)	-59%
Report Length (words)	$\sim$2,800	$\sim$6,800	+143%
Citations	$\sim$15	$\sim$42	+180%
Visual Elements	None	Mermaid diagrams, tables	New capability
Anti-Hallucination	None	Source attribution + contradiction check	New capability
Total Runtime	$\sim$60 s	$\sim$127 s	+112%